

4. Загальні правила роботи

Кожна задача в проєкті проходить однаковий цикл:

1. Issue
- ↓
2. Branch
- ↓
3. Code
- ↓
4. Commit
- ↓
5. Push
- ↓
6. Pull Request
- ↓
7. Code Review
- ↓
8. Corrections
- ↓
9. Merge

Заборонено

Студент не повинен:

- працювати безпосередньо в `main`;
- робити один величезний `commit` наприкінці;
- копіювати код іншого студента;
- змінювати чужу функціональність без узгодження;
- `merge`-ити власний PR без `review`.

Структура repository

Для перших лабораторних достатньо:

```
JavaMathSolver/  
├── README.md  
├── .gitignore  
└── src/  
    └── MathSolver.java
```

Після Lab 3 структура може почати розділятися:

```
src/
├── MathSolver.java
├── algebra/
├── geometry/
└── core/
```

Поки що **не потрібно** передчасно створювати складну архітектуру. Студенти повинні відчувати, чому вона стає необхідною.

Спільне завдання для всіх команд

У кожній лабораторній є **індивідуальні задачі**, але є також один командний milestone.

Lab 1 milestone

MATHSOLVER-0.1

Програма:

- компілюється;
- запускається;
- розуміє команди;
- виконує algebra;
- виконує geometry;
- має help.

Lab 2 milestone

MATHSOLVER-0.2

Програма:

- використовує методи;
 - має switch;
 - має перевірку аргументів;
 - розв'язує перші рівняння;
 - має математичні edge cases.
-

Lab 3 milestone

MATHSOLVER-0.3

Програма:

- працює з масивами;
 - обробляє довільну кількість чисел;
 - виконує статистичні операції;
 - працює з наборами точок;
 - має уніфікований CLI.
-

Git workflow для кожного студента

Крок 1. Отримати Issue

Наприклад:

ALG-14 Implement GCD

Issue повинна містити:

Goal:

Implement `gcd(a, b)`.

Input:

Two integers.

Output:

Greatest common divisor.

Examples:

`gcd 48 18 -> 6`

Edge cases:

`gcd 0 5 -> 5`

Крок 2. Створити branch

```
git checkout main
```

```
git pull
```

```
git checkout -b feature/alg-14-gcd
```

Крок 3. Реалізація

Студент пише код тільки для своєї задачі.

Крок 4. Commit

Commit повинен бути атомарним:

```
git add src/MathSolver.java
git commit -m "feat(algebra): implement gcd"
```

Якщо задача складна, допустимо 2–3 commits:

```
feat(algebra): add gcd algorithm
test(algebra): add gcd edge cases
docs(algebra): document gcd command
```

Крок 5. Push

```
git push -u origin feature/alg-14-gcd
```

Крок 6. Pull Request

PR:

ALG-14 Implement GCD

Description:

What was implemented?

Implemented Euclidean algorithm for GCD.

Example

```
gcd 48 18 -> 6
```

Edge cases

```
gcd 0 5 -> 5
```

Related issue

6. Code Review

Кожен студент повинен зробити review іншого студента.

Review — не:

LGTM

Потрібно перевірити хоча б:

- [] Code compiles
 - [] Correct result
 - [] Edge cases
 - [] Naming
 - [] No unnecessary duplication
 - [] Does not break existing commands
 - [] Matches project conventions
-

7. Cross-team review

Я рекомендую спеціально заборонити схему:

```
Algebra → Algebra  
Geometry → Geometry  
Core → Core
```

Краще:

```
Algebra → Geometry  
Geometry → Core  
Core → Algebra
```

Наприклад:

Algebra student перевіряє geometry PR і повинен зрозуміти:

Чи правильно використана математична формула?

Geometry student перевіряє Core PR:

Чи правильно передаються координати?

Core student перевіряє Algebra PR:

Чи не ламається CLI?

Це дуже добре розвиває командне програмування.

8. Merge policy

PR можна merge тільки якщо:

- ✓ Code compiles
- ✓ Issue solved
- ✓ At least 1 review
- ✓ Review comments resolved
- ✓ No conflict
- ✓ Existing functionality still works

Для важливих інтеграційних PR:

- ✓ 2 reviewers
-

9. Розподіл ролей усередині команди

Ролі ротаційні.

Наприклад:

Lab 1

Algebra
Student A – implementation
Student B – implementation /review
Student C – tests /review
Student D – tests
Student E – docs/PL

Lab 2

ролі змінюються.

Так кожен студент за семестр повинен побувати:

- implementer;
- tester;
- reviewer;
- integrator;
- documentation contributor.

10. Важливе правило: PR має бути маленьким

Поганий PR:

```
Add 15 commands
Refactor entire project
Update README
Fix all bugs
```

Хороший:

```
feat(algebra): implement gcd
```

або:

```
feat(geometry): implement midpoint
```

або:

```
feat(core): validate command arguments
```

Це дозволить на кожній парі реально робити **10–20 маленьких PR**, а не один колективний “Lab 1 complete”.

Лабораторна робота №1

JavaMathSolver 0.1 — перший математичний модуль

Тема

Платформа Java, перший клас, примітивні типи, `String`, форматований вивід, `Math`, аргументи командного рядка.

Це безпосередньо відповідає першому розділу курсу. У ньому також підкреслюється робота з `javac`, `java`, `String[] args`, `Math` та форматованим виводом.

1.1. Мета

Створити **першу версію `JavaMathSolver`**, яка запускається з командного рядка та виконує математичні операції.

Наприклад:

```
java MathSolver add 2 3
```

Результат:

```
2 + 3 = 5
```

1.2. Що студент повинен навчитися

Після лабораторної студент повинен уміти:

- створити `.java` файл;
- оголосити клас;
- написати `main`;
- використовувати `int`, `double`, `boolean`;
- працювати зі `String`;
- отримувати `String[] args`;
- перетворювати `String` у `double`;
- використовувати `Math`;
- виводити результат через `System.out`;
- компілювати програму;
- запускати її через `command line`.

Особливу увагу приділяємо тому, що `args` — це масив рядків, переданих під час запуску програми.

1.3. Загальний контракт програми

На цьому етапі MathSolver підтримує команди:

```
add
sub
mul
div
pow
sqrt
abs
```

Формат:

```
java MathSolver <command> <arguments>
```

1.4. Приклади

Addition

```
java MathSolver add 10 25
10 + 25 = 35
```

Multiplication

```
java MathSolver mul 4 7
4 * 7 = 28
```

Power

```
java MathSolver pow 2 10
2^10 = 1024
```

Square root

```
java MathSolver sqrt 144
sqrt(144) = 12
```

1.5. Завдання команди Algebra

ALG-01 — Addition

Реалізувати:

```
add a b
```

Використати `double`.

ALG-02 — Subtraction

`sub a b`

ALG-03 — Multiplication

`mul a b`

ALG-04 — Division

`div a b`

На цьому етапі необхідно дослідити поведінку Java для:

`10 / 2`
`10.0 / 2`
`1.0 / 0.0`

і пояснити результат.

ALG-05 — Power

`pow a b`

Використати:

`Math.pow(a, b)`

ALG-06 — Absolute value

`abs x`

Використати:

`Math.abs(x)`

1.6. Завдання команди Geometry

Команда Geometry також працює з реальними математичними задачами.

GEO-01 — Distance between points

```
distance x1 y1 x2 y2
```

Формула:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Наприклад:

```
java MathSolver distance 1 2 4 6
```

→

```
distance = 5
```

GEO-02 — Circle area

```
circle-area r  
 $S = \pi r^2.$ 
```

GEO-03 — Circle circumference

```
circle-circumference r  
 $C = 2\pi r.$ 
```

GEO-04 — Rectangle area

```
rectangle-area a b  
 $S = ab.$ 
```

GEO-05 — Rectangle perimeter

```
rectangle-perimeter a b  
 $P = 2(a + b).$ 
```

GEO-06 — Euclidean distance from origin

```
origin-distance x y  
 $d = \sqrt{x^2 + y^2}.$ 
```

1.7. Завдання Core / Text / Integration

Задача — зробити так, щоб результати Algebra і Geometry можна було викликати через одну програму.

CORE-01 — Command detection

Визначити:

```
args[0]
```

як назву операції.

CORE-02 — Argument count

Реалізувати перевірку кількості аргументів.

Наприклад:

```
java MathSolver add 5
```

не повинно призводити до `ArrayIndexOutOfBoundsException`.

CORE-03 — String → double

Опрацювати:

```
Double.parseDouble(...)
```

CORE-04 — Help

Реалізувати:

```
java MathSolver help
```

Результат:

```
JavaMathSolver
```

Available commands:

```
add a b
sub a b
mul a b
div a b
pow a b
sqrt x
```

```
distance x1 y1 x2 y2
circle-area r
circle-circumference r
rectangle-area a b
origin-distance x y
```

CORE-05 — Unknown command

```
java MathSolver hello 1 2
```

повинно дати зрозуміле повідомлення:

```
Unknown command: hello
```

Use 'help' to see available commands.

CORE-06 — README

Створити перший README з:

- описом проєкту;
 - вимогами;
 - способом компіляції;
 - способом запуску;
 - прикладами.
-

1.8. Інтеграційний результат Lab 1

Після merge всіх PR програма повинна підтримувати **мінімум 12 команд**:

```
add
sub
mul
div
pow
abs
```

```
distance
origin-distance
circle-area
circle-circumference
rectangle-area
rectangle-perimeter
```

1.9. Commit policy Lab 1

Рекомендується:

```
feat(algebra): add addition operation
feat(algebra): add multiplication operation
feat(geometry): add point distance
feat(core): add command validation
docs: add initial README
```

Не:

```
final
changes
my code
lab1
asdf
```

Лабораторна робота №1

JavaMathSolver 0.2 — алгоритми та математичні методи

Тема

Вирази, арифметичні й логічні оператори, `if`, `switch`, цикли, статичні методи.

Це відповідає другому розділу курсу: арифметика, логічні вирази, умови, `switch`, цикли та методи.

2.1. Нова вимога

Після Lab 1 програма працює, але код починає ставати поганим.

Уявімо:

```
if (args[0].equals("add")) {  
    ...  
} else if (args[0].equals("sub")) {  
    ...  
} else if (args[0].equals("mul")) {  
    ...  
}
```

Зараз це ще допустимо.

Але якщо команд стане 30?

Тому Lab 2 має навчити студентів **виділяти математичну логіку в методи**.

2.2. Ціль

Перейти від:

```
main()  
└─ весь код
```

до:

```
main()  
├─ визначення команди  
├─ перевірка аргументів  
└─ виклик математичного методу
```

2.3. Algebra — 6 задач

ALG-11 — Linear equation

Розв'язати:

$$ax+b=0$$

Команда:

```
solve-linear a b
```

Приклад:

```
java MathSolver solve-linear 2 -6
```

→

```
x = 3
```

Необхідно обробити:

```
a = 0  
a != 0
```

ALG-12 — Quadratic equation

$ax^2+bx+c=0$

Команда:

```
solve-quadratic a b c
```

Використати дискримінант:

$D=b^2-4ac$

Обробити:

```
D > 0  
D = 0  
D < 0
```

ALG-13 — Maximum of three numbers

```
max3 a b c
```

Заборонено використовувати готову бібліотечну функцію для всієї задачі.

Потрібні `if` / логічні оператори.

ALG-14 — Greatest common divisor

```
gcd a b
```

Реалізувати алгоритм Евкліда.

Це перша задача, де цикл має **математичний зміст**:

ALG-15 — Factorial

`factorial n`

Реалізувати через `for` або `while`.

Обговорити:

- що відбувається для $n < 0$;
- `integer overflow`.

ALG-16 — Fibonacci

`fibonacci n`

Обов'язково реалізувати ітеративний варіант.

Додатково — рекурсивний.

Порівняти їхню ефективність.

2.4. Geometry — 6 задач

GEO-11 — Triangle area

За трьома сторонами:

`a,b,c`

використати формулу Герона:

$S = s(s-a)(s-b)(s-c)$,
 $s = a+b+c/2$

GEO-12 — Triangle validity

`triangle-valid a b c`

Перевірити:

GEO-13 — Point quadrant

quadrant x y

Результат:

I
II
III
IV
AXIS
ORIGIN

Тут активно використовуються `if` та логічні вирази.

GEO-14 — Manhattan distance

$d=|x_2-x_1|+|y_2-y_1|$
manhattan-distance x1 y1 x2 y2

GEO-15 — Midpoint

midpoint x1 y1 x2 y2

GEO-16 — Collinearity

Перевірити, чи лежать три точки на одній прямій.

collinear x1 y1 x2 y2 x3 y3

Наприклад:

collinear 0 0 1 1 2 2

→

true

2.5. Core/Text/Integration — 6 задач

CORE-11 — Command dispatcher

Перейти до `switch` для вибору операції.

Наприклад:

```
switch (command) {  
    case "add":  
        ...  
    case "sqrt":  
        ...  
}
```

CORE-12 — Mathematical argument validation

Створити логіку:

```
command  
↓  
expected number of arguments  
↓  
actual number  
↓  
execute / error
```

CORE-13 — Numeric parsing

Обробити:

```
10  
-5  
3.14  
1e-5
```

CORE-14 — Formatted output

Всі числові результати повинні мати єдиний формат.

Наприклад:

```
Result: 5.000000
```

Використати `System.out.printf`.

CORE-15 — Error messages

Стандартизувати:

```
Unknown command
Invalid number
Wrong number of arguments
Division by zero
Invalid mathematical input
```

CORE-16 — Command documentation

README повинен містити таблицю:

Command	Arguments	Meaning
add	a b	$a+b$
sqrt	x	$\sqrt{x}$
distance	x1 y1 x2 y2	Euclidean distance
solve-linear	a b	$ax+b=0$
...	...	...

2.6. Особливе завдання Lab 2

Mathematical Edge Cases

Кожна команда повинна мати щонайменше:

- нормальний випадок;
- граничний випадок;
- некоректний випадок.

Наприклад:

```
solve-quadratic 1 -5 6
solve-quadratic 1 2 1
solve-quadratic 1 1 1
```

i:

```
solve-quadratic 0 2 4
```

2.7. Результат Lab 2

Версія:

JavaMathSolver 0.2

вже є маленьким математичним CLI.

Вона вміє:

```
arithmetic
+
equations
+
number theory
+
geometry
+
command processing
```